

Cryptography, Theory and Practice CIS 4930/6371 (Fall 2026)

Instructor: Dr. Attila A. Yavuz

TA: Sean Hernandez

Contributor: Sheng Rao

September 24, 2026

Homework 3 (**Due: Oct. 2nd, 2026 – 23 : 59**)[**100 Points**]

(Upload your solutions to the CANVAS)

Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the other is responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner. See syllabus for breaking-up with a programming partner). Directly borrowing (e.g., copy-paste) from any material and putting it in your solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations.**

Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also do research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially not be sufficient.

Remark: Answers to questions should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.

As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

1 Programming Assignment 1: Encryption/Decryption with AES-CTR utilizing hash chains [**40 pt**]

In this exercise we will implement encryption/decryption through AES-CTR while using hash chains to fulfill the key requirements of AES-CTR, as seen in “SymmetricCrypto.pptx”, slide 49. The plaintext is divided into 512-bit blocks, and each block requires its own key to encrypt/decrypt. Rather than distribute a large number of keys, we can just distribute a seed and use a hash chain to derive as many keys from it as we need, in a PRNG-like fashion.

Alice will do the encryption portion, while Bob will do the decryption portion.

Alice:

1. Alice reads the shared seed from the file “**SharedSeed.txt**”, and the list of messages from the file “**Messages.txt**”.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).

2. Alice creates the first key to be used in the encryption by hashing the shared seed with SHA256.
3. For each line extracted from “Messages.txt”, Alice encrypts the line with the key, then computes the next key to be used by hashing the just-used key with SHA256, forming a hash chain.
4. Alice writes all the keys used (in Hex string format) to a file named “**Keys.txt**”, with each key being a single line in the file.
5. Alice writes the encrypted messages in Hex string format to a file named “**Ciphertexts.txt**”, with each encrypted message being a single line in the file.

Bob:

1. Bob reads the shared seed from the file “**SharedSeed.txt**”, and the list of ciphertexts (in Hex string format) from the file “**Ciphertexts.txt**”.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. Bob creates the first key to be used in the decryption by hashing the shared seed with SHA256.
3. For each line extracted from “Ciphertexts.txt”, Bob decrypts the line with the key, then computes the next key to be used by hashing the just-used key with SHA256, forming a hash chain.
4. Bob writes the decrypted messages to a file named “**Plaintexts.txt**”, with each decrypted message being a single line in the file. The plaintexts should match the original messages in “Messages.txt”.

Programming Assignment 1 - Specific Notes

- In “Messages.txt”, each line is a string of exactly 64 characters, or 64 bytes / 512 bits, with the trailing newline to be ignored/stripped. The number of lines is set and can be referenced from the `NUMBER_OF_MESSAGES` constant in `RequiredFunctionsHW3.c`.
- Compile your code with the following commands:
`gcc alice.c -lssl -lcrypto -o alice`
`gcc bob.c -lssl -lcrypto -o bob`
- Run your code with the first test files with the following commands:
`./alice SharedSeed1.txt Messages1.txt`
`./bob SharedSeed1.txt Ciphertexts.txt`
- We have provided a script (`VerifyingChainCrypt.sh`) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put “alice.c”, “bob.c”, the provided test vectors, and the “VerifyingChainCrypt.sh” script in one folder and run the following command in the terminal:
`bash VerifyingChainCrypt.sh`
- The verification script uses the same format as the one that is run to grade your submissions, with different test vectors.
- **You Must submit 2 files for this part of the assignment:**
`alice.c` & `bob.c`

Rubric:

- The “**Keys.txt**” file **[10 pt]**
- The “**Ciphertexts.txt**” file **[15 pt]**
- The “**Plaintexts.txt**” file **[15 pt]**

2 Programming Assignment 2: Davies-Meyer compression function [20 pt]

In this exercise we will implement the Davies-Meyer construction of a one-way compression function. Davies-Meyer construction uses a block cipher to iterate hashing over inputs of arbitrary size (as long as the input is padded to a multiple of the block size used in the block cipher), creating a collision-resistant one-way compression function with full avalanche effect.

In our case, the block cipher we use is AES128-ECB, which has 128-bit (16-byte) block size and 128-bit output, and while it is not cryptographically secure since it does not use chaining or IV's, this simplicity makes it useful as a cryptographic primitive for toy implementations. The Davies-Meyer method is to compute the 16-byte hash output of a message block as an encryption of the message block with the key being the hash output of the previous block, then XOR'd with the hash output of the previous block, specifically $Hash_i = AES(Msg_i, Hash_{i-1}) \oplus Hash_{i-1}$. Repeat the computation until all message blocks have been run through the process, with the final Hash as the output/result of this process.

Davies-Meyer:

1. The program reads the message from the file **"Message.txt"**
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The program divides the message into 16-byte blocks, padding the ending with zeroes if it is not enough to fill a block.
3. For each block, the program computes the hash of that block as $Hash_i = AES(Msg_i, Hash_{i-1}) \oplus Hash_{i-1}$, where the AES function is in the format AES(plaintext, key).
4. The program writes the result of the first hash (the hash result of the first message block) in Hex string format to a file named **"FirstHash.txt"**.
5. The program writes the result of the final hash (the hash result of the last message block) in Hex string format to a file named **"FinalHash.txt"**.

Programming Assignment 2 - Specific Notes

- For the first hash of the first message block, since there is no "previous hash" or $Hash_{i-1}$, use a block with bits initialized to all zeroes as the previous hash.
- In grading at our side, the message and length of the message will be chosen at random and everything must work.
- Compile your code with the following commands:
gcc dm.c -lssl -lcrypto -o dm
- Run your code with the first test files with the following command:
./dm Message1.txt
- We have provided a script (VerifyingDaviesMeyer.sh) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put "dm.c" and the rest of the provided files along with the "VerifyingDavies-Meyer.sh" script in one folder and run the following command in the terminal:
bash VerifyingDaviesMeyer.sh

- The verification script uses the same format as the one that is run to grade your submissions, with different test vectors.
- **You Must submit 1 file for this part of the assignment:**
dm.c

Rubric:

- The “**FirstHash.txt**” file **[10 pt]**
- The “**FinalHash.txt**” file **[10 pt]**

3 Programming Assignment 3: Shamir Secret Sharing **[40 pt]**

In this exercise we will put into action the Shamir Secret Sharing scheme as described in “ShamirSecretSharing.pptx”, slides 4-7, inside a finite arithmetic field (with selected prime modulus). With Shamir Secret Sharing one can divide a secret into multiple parts (shares), such that only a specific number of shares (threshold) are required to reconstruct the secret. It is based on polynomial interpolation: the secret is embedded as the constant term of a randomly generated polynomial of degree $t - 1$ where t is the chosen threshold, and each share is a point on this polynomial. Because fewer than t points provide no information about the polynomial, this scheme is both secure and fault-tolerant.

Share Generator:

1. The Generator reads the secret and the prime modulus (in decimal string format) from the files “**Secret.txt**” and “**Modulus.txt**” respectively
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The Generator randomly creates the coefficients for the polynomial (with the secret being the constant factor), then creates the specified number of shares as points on the generated polynomial.
3. The Generator writes the shares to files “**Share1.txt**” to “**ShareX.txt**”, where X is the number of shares to be generated. So for three shares to be generated, you would write to Share1.txt, Share2.txt, and Share3.txt. Each share contains two lines, the first being the X-value of the point and the second being the Y-value of the point. Each value should be written in decimal string format.

Share Reconstructor:

1. The Reconstructor reads the prime modulus and a (threshold) number of shares (in decimal string format) from the files “**Modulus.txt**” and the Share files generated by the Generator respectively
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The Reconstructor utilizes Lagrange polynomial interpolation to recover the constant factor of the polynomial indicated by the shares, which is the secret.
3. The Reconstructor writes the recovered secret in decimal string format to a file named “**Recovered.txt**”.

Programming Assignment 3 - Specific Notes

- Please use type `uint64_t` for your integers so operations with 32-bit integers don't overflow.
- The files store the number as decimal strings, so, for example, reading “100” from a file gives you the string “100”, which represents the integer 100. Likewise the file output of the results should also be in decimal string format, so a resulting integer of 99 should be written as the string “99” to the file.

- With `uint64_t`, care must be taken with subtraction operations to avoid working with negative integers, thankfully operations being inside a modulus gives you an easy way to convert negative numbers to positive ones.
- To avoid floating point imprecision as a result of division, you can leverage modular arithmetic to multiply by the modular inverse of a number instead of dividing by it. You can re-use your modular inverse and modular exponentiation code from previous assignments.
- In grading at our side, the secret and prime modulus numbers will be chosen at random (but within 32 bit length) and everything must work.
- Compile your code with the following commands:
`gcc shamirgen.c -lssl -lcrypto -o gen`
`gcc shamirrecon.c -lssl -lcrypto -o recon`
- Run your code with the first test files with the following command:
`./gen Secret1.txt Modulus1.txt`
`./recon Modulus1.txt`
- We have provided a script (`VerifyingShamir.sh`) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- This testing script and vectors package also includes a pre-compiled reconstruction binary file named “reconalt” that randomly selects a (threshold) number of shares and recovers the secret from them. Its behavior should be basically the same as a correctly coded `shamirrecon.c` (with perhaps a wider testing of shares). “reconalt” is used by the verifying script to verify the correctness of the generated shares, and it may not work correctly on architectures different from the grader’s machine but it will work correctly on the grader’s machine where it was compiled.
- Please name the shares according to the specified convention (`Share1.txt`, `Share2.txt` ...), otherwise the Verification script will not be able to check the correctness of those shares.
- In order to use the script, just put “`shamirgen.c`”, “`shamirrecon.c`”, and the rest of the provided files along with the “`VerifyingShamir.sh`” script in one folder and run the following command in the terminal:
`bash VerifyingShamir.sh`
- The verification script is the same as the one that is run to grade your submissions, only with different test vectors.
- **You Must submit 2 files for this part of the assignment:**
`shamirgen.c` & `shamirrecon.c`

Rubric:

- The “`Recovered.txt`” file [20 pt]
- Passes the grader’s reconstruction verification run on the generated shares [20 pt]

4 Extra Credit Question and Answer

4.1 Shamir Secret Sharing and Error Correcting Codes [+10 pt Extra Credit]

Write a 3 paragraph response on the connection between Shamir secret sharing and error correcting codes:

- Explain why the vector of n Shamir shares of a degree- $(k - 1)$ polynomial is a codeword of a Reed-Solomon code. What does reconstructing the secret from k shares correspond to in coding-theory terms? [4 pt]

- B. Suppose some shareholders submit incorrect shares. Using the error-correcting properties of Reed-Solomon codes, how many corrupted shares can be tolerated (as a function of n and k) while still recovering the correct secret? [3 pt]
- C. Give at least two examples of where this connection is used in practice (e.g., in secure multiparty computation, robust secret sharing, or distributed storage). [3 pt]

4.2 Extra Credit: Byzantine Fault Tolerance [+10 pt Extra Credit]

- A. Describe in a paragraph or two what Byzantine Fault Tolerance is and why at least $3f + 1$ replicas are needed to tolerate f Byzantine replicas in the classical (partially synchronous) setting. Give an example of how BFT is used to keep a replicated database consistent. [4 pt]
- B. What role do cryptographic primitives (e.g., digital signatures, MACs, or hash functions) play in BFT protocols such as PBFT? What could a Byzantine node do if messages were not authenticated? [3 pt]
- C. How does Proof-of-Work relate to classical BFT? Compare the two in terms of who participates, what fraction of adversarial power each tolerates, and whether finality is deterministic or probabilistic. [3 pt]

Resources: Here are some resources that may be useful when answering these extra credit questions:

- **McEliece and Sarwate, "On Sharing Secrets and Reed-Solomon Codes," 1981**
<https://doi.org/10.1145/358746.358762>
- **Error-Correcting Codes - Cryptography 101 with Alfred Menezes**
<https://cryptography101.ca/codes/>
- **Kartik Nayak, Ittai Abraham, "Practical Byzantine Fault Tolerant Consensus (PBFT)"**
<https://decentralizedthoughts.github.io/2025-02-14-PBFT/>

You must submit 1 file for extra credit part of this assignment: **HW3Group(YourGroupNumber).pdf**

Common Assignment Notes

- For the correctness of reading from a file, please use unsigned char.
- For your convenience, we have provided a template file with the necessary functions and descriptions:
RequiredFunctionsHW3.c
- You may use the functions in RequiredFunctionsHW3.c, modify them, or write your own, modifying the RequiredFunctionsHW3.c file as necessary. **If you modify it, submit your modified RequiredFunctionsHW3.c file alongside your other code files in Canvas.**
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn't matter if it works on your computer!).
- **Follow the Instructions and all the specific names of output files mentioned above, otherwise there will be 20% deduction for inconvenience.**